Sesión 4: ARQUITECTURA, OPTIMIZACIÓN Y MODELAMIENTO AVANZADO DE BASES DE DATOS

Del diseño a la implementación en entornos reales

Por
Carlos César Sánchez Coronel

2026

Índice general

Índice general **2**

1. Introducción **3**

2. Arquitecturas de Bases de Datos **4**
2.1. Modelo Centralizado vs. Cliente-Servidor 4
2.2. Bases de Datos Distribuidas 4
2.2.1. Fragmentación 4
2.2.2. Replicación 5
2.2.3. Transparencia 5
2.3. Clustering y Alta Disponibilidad 5
2.4. Sharding 5
2.5. Bases de Datos en la Nube (DBaaS) 6

3. Optimización de Consultas y Uso de Índices **7**
3.1. El Optimizador de Consultas 7
3.2. Planes de Ejecución 7
3.3. Estadísticas 8
3.4. Tipos de Índices Avanzados 8
3.5. Estrategias de Indexación 8
3.6. Ejercicio Resuelto: Optimización de una Consulta Lenta 8

4. Transacciones y Concurrencia **10**
4.1. Propiedades ACID 10
4.2. Niveles de Aislamiento 10
4.3. Implementación de Concurrencia 11
4.3.1. Bloqueos (Locks) 11
4.3.2. MVCC (Multi-Version Concurrency Control) 11
4.3.3. Deadlocks 11
4.4. Ejercicio Resuelto: Simulación de Concurrencia 11

5. Integridad Avanzada y Programación en Base de Datos **12**
5.1. Restricciones Avanzadas 12
5.2. Triggers 12
5.3. Procedimientos Almacenados y Funciones 13
5.4. Vistas Materializadas 13
5.5. Ejercicio Resuelto: Auditoría con Trigger 13

6. Data Warehousing y Modelamiento Dimensional **15**
6.1. OLTP vs OLAP . . . 15
6.2. Esquema Estrella (Star Schema) . . . 15
6.3. Esquema Copo de Nieve (Snowflake) . . . 16
6.4. Tipos de Tablas de Hechos . . . 16
6.5. Dimensiones Lentamente Cambiantes (SCD) . . . 16
6.6. Procesos ETL/ELT . . . 16
6.7. Ejercicio Resuelto: Diseño de un Data Warehouse de Ventas . . . 16

7. Introducción a NoSQL **18**
7.1. Motivación . . . 18
7.2. Clasificación . . . 18
7.3. Teorema CAP . . . 18
7.4. Modelamiento en MongoDB . . . 19
7.5. Modelamiento en Cassandra . . . 19
7.6. Cuándo usar NoSQL y cuándo SQL . . . 20
7.7. Ejercicio Resuelto: Comparación de Modelamiento . . . 20

8. Casos Prácticos Integradores (Ejercicios Resueltos) **21**
8.1. Ejercicio 1: Banca en Línea . . . 21
8.2. Ejercicio 2: Data Warehouse de Ventas . . . 22
8.3. Ejercicio 3: Trigger de Histórico . . . 22
8.4. Ejercicio 4: Comparación SQL vs MongoDB . . . 23

Glosario **24**

Referencias **25**

Capítulo 1

Introducción

En la Sesión 3 abordamos los fundamentos del modelamiento de datos, desde el nivel conceptual hasta el físico, incluyendo normalización e índices. En esta Sesión 4, daremos un paso adelante para explorar aspectos avanzados que son cruciales en entornos profesionales: arquitecturas de bases de datos, optimización de consultas, manejo de transacciones y concurrencia, integridad avanzada mediante programación en la base de datos, modelamiento dimensional para data warehousing, y una introducción a las bases de datos NoSQL. Cada capítulo incluye ejemplos prácticos y ejercicios resueltos que te ayudarán a consolidar los conceptos y a prepararte para enfrentar desafíos reales, así como preguntas técnicas en entrevistas nacionales e internacionales.

Capítulo 2

Arquitecturas de Bases de Datos

La arquitectura de una base de datos define cómo se organizan los componentes de software y hardware para almacenar, procesar y acceder a los datos. La elección de la arquitectura impacta directamente en la escalabilidad, disponibilidad y rendimiento.

2.1. Modelo Centralizado vs. Cliente-Servidor

En el modelo **centralizado**, todos los datos residen en un único servidor y los usuarios se conectan a través de terminales. Es simple pero poco escalable y vulnerable a fallos.

El modelo **cliente-servidor** separa la lógica de presentación (cliente) de la gestión de datos (servidor). Los clientes envían solicitudes SQL al servidor, que procesa y devuelve resultados. Este modelo es la base de la mayoría de los SGBD actuales (PostgreSQL, MySQL, Oracle).

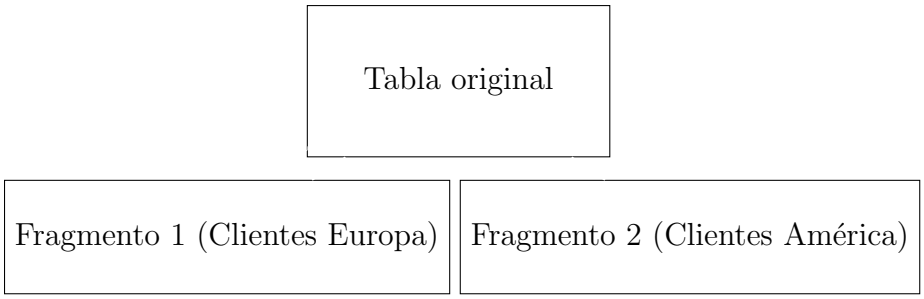
2.2. Bases de Datos Distribuidas

Una base de datos distribuida almacena datos en múltiples nodos interconectados, ofreciendo transparencia al usuario. Puede ser homogénea (todos los nodos usan el mismo SGBD) o heterogénea.

2.2.1. Fragmentación

La fragmentación divide una tabla en fragmentos (particiones) que se almacenan en diferentes nodos. Tipos:

- **Horizontal:** Filas completas se distribuyen según un criterio (ej. clientes por región). Cada fragmento tiene las mismas columnas.
- **Vertical:** Columnas se distribuyen en diferentes fragmentos, manteniendo la clave primaria en todos.
- **Híbrida:** Combinación de ambas.



2.2.2. Replicación

La replicación mantiene copias idénticas de los datos en varios servidores. Puede ser:

- **Síncrona:** Todas las copias se actualizan en la misma transacción. Alta consistencia, pero mayor latencia.
- **Asíncrona:** La actualización se propaga después de confirmar la transacción. Menor consistencia, pero mejor rendimiento.

La replicación mejora la disponibilidad y permite balanceo de carga de lecturas.

2.2.3. Transparencia

En un sistema distribuido ideal, el usuario no percibe la distribución. Existen varios niveles de transparencia: de localización, de fragmentación, de replicación, etc.

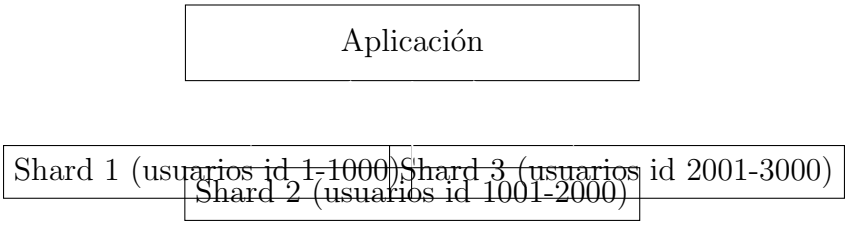
2.3. Clustering y Alta Disponibilidad

Un **cluster** es un grupo de servidores que trabajan juntos para ofrecer alta disponibilidad y escalabilidad. Configuraciones típicas:

- **Active-Passive:** Un nodo activo procesa solicitudes; el pasivo está en espera para tomar el control si el activo falla.
- **Active-Active:** Todos los nodos procesan solicitudes simultáneamente, requiriendo mecanismos de coordinación (ej. bloqueos distribuidos).

2.4. Sharding

El **sharding** es una técnica de particionamiento horizontal a gran escala, donde cada fragmento (shard) es una base de datos independiente. Los shards se distribuyen en múltiples servidores. La clave de partición determina en qué shard se almacena cada fila. Es común en aplicaciones web masivas (ej. redes sociales, juegos en línea).



2.5. Bases de Datos en la Nube (DBaaS)

Las plataformas como servicio de bases de datos (DBaaS) ofrecen bases de datos gestionadas en la nube (Amazon RDS, Google Cloud SQL, Azure SQL Database). Ventajas: escalabilidad automática, backups gestionados, alta disponibilidad integrada, pago por uso.

Capítulo 3

Optimización de Consultas y Uso de Índices

La optimización es clave para garantizar un rendimiento aceptable, especialmente con grandes volúmenes de datos.

3.1. El Optimizador de Consultas

El SGBD incluye un optimizador que decide el plan de ejecución más eficiente para una consulta SQL. Puede ser:

- **Basado en reglas:** Usa heurísticas (ej. siempre usar índices si existen).
- **Basado en costos:** Estima el costo de diferentes planes (E/S, CPU, memoria) y elige el de menor costo. Es el más común (PostgreSQL, Oracle, SQL Server).

3.2. Planes de Ejecución

El plan de ejecución muestra las operaciones que realizará el motor. Operadores comunes:

- **Table Scan (Seq Scan):** Recorre toda la tabla. Costo $O(n)$.
- **Index Scan:** Usa un índice para localizar filas rápidamente.
- **Index Only Scan:** Si el índice contiene todas las columnas necesarias, no accede a la tabla.
- **Nested Loop:** Para cada fila de una tabla, busca coincidencias en la otra usando un índice. Eficiente para conjuntos pequeños.
- **Hash Join:** Construye una tabla hash en memoria con una de las tablas y luego la recorre con la otra. Bueno para tablas sin índices.
- **Merge Join:** Ordena ambas tablas y luego las fusiona. Útil si ya están ordenadas.

```
-- Ver el plan de ejecuci n en PostgreSQL
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM clientes WHERE apellido
   = 'Garc a';
```

3.3. Estadísticas

El optimizador usa estadísticas sobre las tablas (número de filas, distribución de valores, histogramas) para estimar la selectividad de las condiciones. Es crucial mantener las estadísticas actualizadas (`ANALYZE` en PostgreSQL).

3.4. Tipos de Índices Avanzados

Además de los índices B-Tree, existen otros tipos útiles:

- **Índice cubriente:** Incluye todas las columnas necesarias para una consulta, evitando acceder a la tabla. En PostgreSQL se logra con índices que incluyen columnas adicionales (`INCLUDE`).
- **Índice funcional:** Indexa el resultado de una función, ej. `INDEX ON (LOWER(email))`.
- **Índice parcial:** Solo indexa un subconjunto de filas que cumplen una condición, ahorrando espacio. Ej. `WHERE activo = true`.
- **Índice con columnas incluidas:** En SQL Server, se pueden incluir columnas no clave en el índice para hacerlo cubriente.

3.5. Estrategias de Indexación

No existe una fórmula única; hay que analizar las consultas más frecuentes y críticas. Reglas generales:

- Indexar columnas usadas en `WHERE`, `JOIN`, `ORDER BY`.
- Para consultas con múltiples condiciones, considerar índices compuestos con el orden adecuado (primero las columnas más selectivas).
- Evitar índices en columnas con pocos valores distintos (baja cardinalidad) a menos que se usen combinados.
- Monitorizar índices no utilizados y eliminarlos.

3.6. Ejercicio Resuelto: Optimización de una Consulta Lenta

Problema: La consulta `SELECT * FROM ventas WHERE fecha BETWEEN '2025-01-01' AND '2025-01-31' ORDER BY total DESC` tarda varios segundos en una tabla de 10 millones de filas. No hay índices.

Solución paso a paso:

1. Analizamos el plan: seguramente es un Seq Scan y luego un Sort.
2. Creamos un índice compuesto por (fecha, total) para cubrir el filtro y el orden:

```sql
CREATE INDEX idx_ventas_fecha_total ON ventas(fecha, total
   DESC);
```

3. El índice permite un Index Scan que devuelve las filas ya ordenadas, evitando el Sort.

4. Si la consulta solo necesita algunas columnas, podríamos crear un índice cubriente incluyendo esas columnas.

Capítulo 4

Transacciones y Concurrencia

Las transacciones garantizan que las operaciones se ejecuten de manera fiable y aislada.

4.1. Propiedades ACID

- **Atomicidad:** La transacción se ejecuta completamente o no se ejecuta. Se implementa con logs de undo/redo.
- **Consistencia:** Las restricciones de integridad se mantienen antes y después.
- **Aislamiento:** Las transacciones concurrentes no interfieren. Se logra mediante bloqueos o MVCC.
- **Durabilidad:** Una vez confirmada, la transacción persiste incluso ante fallos. Se usa write-ahead logging (WAL).

4.2. Niveles de Aislamiento

El estándar SQL define cuatro niveles, de menor a mayor aislamiento:

1. **Read Uncommitted:** Permite leer datos no confirmados (dirty reads). Prácticamente no se usa.
2. **Read Committed:** Solo lee datos confirmados. Evita dirty reads, pero pueden ocurrir non-repeatable reads (una fila leída dos veces puede cambiar si otra transacción la modifica y confirma).
3. **Repeatable Read:** Asegura que si se lee una fila dos veces, su valor no cambia (evita non-repeatable reads). Puede haber phantom reads (aparición de nuevas filas que cumplen la condición).
4. **Serializable:** Máximo aislamiento, las transacciones se ejecutan como si fueran secuenciales. Evita todos los fenómenos.

Nivel	**Dirty Read**	**Non-repeatable Read**	**Phantom Read**	**Serialization An**
Read Uncommitted	Posible	Posible	Posible	Posible
Read Committed	No	Posible	Posible	Posible
Repeatable Read	No	No	Posible	Posible
Serializable	No	No	No	No

4.3. Implementación de Concurrencia

4.3.1. Bloqueos (Locks)

Los SGBD usan bloqueos para controlar el acceso concurrente:

- **Compartidos (Shared):** Para lectura. Varias transacciones pueden tener bloqueos compartidos sobre el mismo recurso.
- **Exclusivos (Exclusive):** Para escritura. Solo una transacción puede tenerlo.
- **Intencionales:** Indican la intención de adquirir bloqueos a nivel más fino (ej. bloqueo intencional compartido a nivel de tabla antes de bloquear filas).

4.3.2. MVCC (Multi-Version Concurrency Control)

MVCC es una técnica utilizada por PostgreSQL, Oracle, etc. Cada transacción ve una instantánea (snapshot) de los datos en un momento dado. Las modificaciones crean nuevas versiones de las filas, sin bloquear lecturas. Así se logra un alto grado de concurrencia.

4.3.3. Deadlocks

Un deadlock ocurre cuando dos o más transacciones se bloquean mutuamente. Ejemplo: T1 bloquea A y espera B; T2 bloquea B y espera A. El SGBD detecta deadlocks y mata una de las transacciones (rollback) para resolverlo.

4.4. Ejercicio Resuelto: Simulación de Concurrencia

Problema: En PostgreSQL, dos transacciones intentan actualizar la misma fila simultáneamente. ¿Qué ocurre según el nivel de aislamiento?

Solución:

1. En `READ COMMITTED` (por defecto en PostgreSQL), la segunda transacción que intenta actualizar espera hasta que la primera confirme o cancele. Si la primera confirma, la segunda ve el nuevo valor y procede.
2. En `REPEATABLE READ`, si la primera confirma, la segunda obtendrá un error de serialización (no puede actualizar porque su snapshot no coincide). Debe reintentar.

Código de ejemplo:

```
-- Sesi n 1
BEGIN;
UPDATE cuenta SET saldo = saldo - 100 WHERE id = 1;
-- (no commit a n)

-- Sesi n 2
BEGIN;
UPDATE cuenta SET saldo = saldo + 100 WHERE id = 1; -- Esto se
   bloquea hasta que Sesi n 1 haga commit o rollback.
```

Capítulo 5

Integridad Avanzada y Programación en Base de Datos

Más allá de las restricciones básicas, los SGBD ofrecen mecanismos para implementar lógica de negocio directamente en la base de datos.

5.1. Restricciones Avanzadas

- **CHECK con subconsultas:** En algunos SGBD se pueden incluir subconsultas, aunque no siempre es recomendable por rendimiento.
- **Acciones referenciales:** ON DELETE CASCADE, ON UPDATE SET NULL, etc., mantienen la integridad referencial automáticamente.

5.2. Triggers

Un trigger es un procedimiento que se ejecuta automáticamente ante un evento (INSERT, UPDATE, DELETE) sobre una tabla. Puede ser BEFORE, AFTER o INSTEAD OF.

Ejemplo: Crear un trigger que audite los cambios de salario en una tabla empleados.

```
CREATE TABLE audit_salario (
    id_empleado INT,
    salario_anterior DECIMAL,
    salario_nuevo DECIMAL,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    usuario TEXT DEFAULT current_user
);

CREATE OR REPLACE FUNCTION audit_salario_func()
RETURNS TRIGGER AS $$
BEGIN
    IF OLD.salario IS DISTINCT FROM NEW.salario THEN
        INSERT INTO audit_salario (id_empleado, salario_anterior,
            salario_nuevo)
        VALUES (OLD.id, OLD.salario, NEW.salario);
    END IF;
```

```
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trig_audit_salario
AFTER UPDATE OF salario ON empleados
FOR EACH ROW EXECUTE FUNCTION audit_salario_func();
```

5.3. Procedimientos Almacenados y Funciones

Permiten encapsular lógica compleja en la base de datos, reduciendo el tráfico de red y centralizando reglas de negocio. Se escriben en PL/pgSQL (PostgreSQL), T-SQL (SQL Server), etc.

Ejemplo: Función que devuelve los empleados de un departamento.

```
CREATE FUNCTION empleados_por_depto(depto_id INT)
RETURNS TABLE(id INT, nombre TEXT) AS $$
BEGIN
    RETURN QUERY SELECT id, nombre FROM empleados WHERE
       departamento_id = depto_id;
END;
$$ LANGUAGE plpgsql;
```

5.4. Vistas Materializadas

Una vista materializada almacena físicamente el resultado de una consulta, a diferencia de una vista normal que es solo una definición. Útil para reportes que no requieren datos en tiempo real. Se pueden refrescar periódicamente.

```
CREATE MATERIALIZED VIEW resumen_ventas AS
SELECT producto_id, SUM(cantidad) as total_vendido
FROM ventas
GROUP BY producto_id;

-- Refrescar
REFRESH MATERIALIZED VIEW resumen_ventas;
```

5.5. Ejercicio Resuelto: Auditoría con Trigger

Problema: Implementar un trigger que registre en una tabla `log`$_e liminacionescadavezqueseborreun$

Solución:

```
CREATE TABLE log_eliminaciones (
    id_cliente INT,
    nombre VARCHAR(100),
    email VARCHAR(100),
    fecha_eliminacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    usuario TEXT DEFAULT current_user
```

```
);

CREATE OR REPLACE FUNCTION log_eliminar_cliente()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO log_eliminaciones (id_cliente, nombre, email)
    VALUES (OLD.id, OLD.nombre, OLD.email);
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trig_log_eliminar
BEFORE DELETE ON clientes
FOR EACH ROW EXECUTE FUNCTION log_eliminar_cliente();
```

Capítulo 6

Data Warehousing y Modelamiento Dimensional

El data warehousing se centra en el análisis de grandes volúmenes de datos históricos, a diferencia de los sistemas OLTP (transaccionales).

6.1. OLTP vs OLAP

- OLTP (On-Line Transaction Processing): Muchas transacciones cortas, consultas simples, datos normalizados.
- OLAP (On-Line Analytical Processing): Consultas complejas de agregación, datos desnormalizados (esquema estrella), históricos.

6.2. Esquema Estrella (Star Schema)

Consta de una tabla de hechos rodeada de tablas de dimensiones. La tabla de hechos contiene medidas numéricas y claves foráneas a las dimensiones. Las dimensiones describen los ejes de análisis (tiempo, producto, cliente, etc.).

6.3. Esquema Copo de Nieve (Snowflake)

Las dimensiones se normalizan en múltiples tablas (ej. dimensión producto se divide en producto, categoría, proveedor). Ahorra espacio pero complica las consultas.

6.4. Tipos de Tablas de Hechos

- **Transaccionales:** Cada fila representa una transacción (ej. venta individual).
- **Periódicas:** Resumen de un período (ej. ventas diarias por producto).
- **Acumulativas:** Para procesos de flujo (ej. pedido desde que se crea hasta que se entrega).

6.5. Dimensiones Lentamente Cambiantes (SCD)

Las dimensiones pueden cambiar con el tiempo. Se definen tipos:

- **Tipo 1:** Sobrescribir el valor anterior (no hay histórico).
- **Tipo 2:** Crear una nueva fila con nuevo rango de vigencia (se mantiene histórico).
- **Tipo 3:** Agregar columnas para valores anteriores y actuales (limitado).

6.6. Procesos ETL/ELT

Extraer, Transformar y Cargar (ETL) o Extraer, Cargar y Transformar (ELT) son procesos que mueven datos desde fuentes operacionales al data warehouse. Herramientas comunes: Apache Spark, Talend, Pentaho, Informatica.

6.7. Ejercicio Resuelto: Diseño de un Data Warehouse de Ventas

Problema: Diseñar un data warehouse para analizar ventas de una tienda online. Se requiere conocer las ventas por producto, cliente, fecha y sucursal.

Solución:

1. Identificamos las dimensiones: Tiempo, Producto, Cliente, Sucursal.
2. Tabla de hechos: Ventas con medidas: cantidad, precio, importe total.
3. Granularidad: cada fila corresponde a una línea de pedido.
4. Esquema estrella:
   - dim_tiempo (id_tiempo, fecha, dia, mes, año, trimestre)
   - dim_producto (id_producto, nombre, categoria, precio_actual)
   - dim_cliente (id_cliente, nombre, ciudad, segmento)

- dim_sucursal (id_sucursal, nombre, ciudad)
- hechos_ventas (id_venta, id_tiempo, id_producto, id_cliente, id_sucursal, cantidad, precio, importe)

5. Si queremos histórico de precios, podríamos incluir precio en hechos (ya que puede cambiar).

Capítulo 7

Introducción a NoSQL

Las bases de datos NoSQL surgieron para manejar grandes volúmenes de datos no estructurados o semiestructurados, con esquemas flexibles y escalabilidad horizontal.

7.1. Motivación

- Escalabilidad horizontal (sharding nativo).
- Modelos de datos flexibles (sin esquema fijo).
- Alto rendimiento para ciertos patrones de acceso.
- Disponibilidad y tolerancia a particiones (teorema CAP).

7.2. Clasificación

- **Clave-Valor:** Redis, DynamoDB. Almacenan pares clave-valor, acceso por clave.
- **Documentales:** MongoDB, Couchbase. Almacenan documentos JSON, permiten consultas por atributos.
- **Columnares:** Cassandra, HBase. Organizan datos en columnas, optimizadas para consultas de agregación.
- **Grafos:** Neo4j, Amazon Neptune. Nodos y relaciones con propiedades, ideales para redes sociales, recomendaciones.

7.3. Teorema CAP

En sistemas distribuidos, no se pueden garantizar simultáneamente Consistencia, Disponibilidad y Tolerancia a Particiones. Los sistemas NoSQL eligen dos:

- **CP:** Consistencia y tolerancia a particiones (ej. HBase). Pueden sacrificar disponibilidad.
- **AP:** Disponibilidad y tolerancia a particiones (ej. Cassandra, DynamoDB). Pueden ser eventualmente consistentes.

- **CA:** Consistencia y disponibilidad, pero no toleran particiones (sistemas tradicionales).

7.4. Modelamiento en MongoDB

MongoDB almacena documentos BSON (JSON binario). El modelamiento se basa en las consultas: se desnormaliza para evitar joins (que no existen). Patrones comunes:

- **Documentos anidados:** Si la relación es uno a pocos y los datos se consultan juntos (ej. direcciones de un cliente).
- **Referencias:** Si los datos son compartidos o muy grandes, se guarda el ID de otro documento (como una FK, pero sin integridad automática).

Ejemplo: Blog con posts y comentarios. En SQL serían dos tablas. En MongoDB:

```
{
  _id: 1,
  titulo: "Mi post",
  contenido: "...",
  comentarios: [
    { autor: "Juan", texto: "Genial", fecha: "2025-03-01" },
    { autor: "Ana", texto: "Gracias", fecha: "2025-03-02" }
  ]
}
```

7.5. Modelamiento en Cassandra

Cassandra usa un modelo de tablas pero con un diseño orientado a consultas. La clave primaria se compone de:

- **Clave de partición:** determina el nodo donde se almacena la fila.
- **Clustering columns:** ordenan las filas dentro de la partición.

Las consultas deben incluir la clave de partición; de lo contrario, son ineficientes (scans). Se desnormaliza para evitar joins.

Ejemplo: Aplicación de mensajería. Consulta: obtener los mensajes de un usuario ordenados por fecha.

```
CREATE TABLE mensajes (
    usuario_id UUID,
    fecha TIMESTAMP,
    mensaje TEXT,
    PRIMARY KEY ((usuario_id), fecha)
) WITH CLUSTERING ORDER BY (fecha DESC);
```

7.6. Cuándo usar NoSQL y cuándo SQL

- **SQL:** Cuando se requieren transacciones ACID, consultas complejas con joins, integridad referencial, esquema estable.
- **NoSQL:** Cuando se necesita escalabilidad horizontal masiva, esquema flexible, alta disponibilidad, y se puede tolerar consistencia eventual.

7.7. Ejercicio Resuelto: Comparación de Modelamiento

Problema: Modelar un sistema de e-commerce con productos, clientes, pedidos y líneas de pedido en SQL y MongoDB.

Solución SQL: Tablas normalizadas (producto, cliente, pedido, detalle_pedido).

Solución MongoDB: Podemos tener una colección `pedidos` que contenga un documento con datos del cliente (embebidos o referenciados) y un array de productos con cantidad y precio. Si el cliente puede cambiar su dirección, conviene referenciarlo. Ejemplo:

```
{
  _id: 1001,
  fecha: "2025-03-15",
  cliente: { id: 1, nombre: "Juan", direccion: "..." }, //
     embebido si no cambia frecuentemente
  lineas: [
    { producto_id: 10, nombre: "Laptop", cantidad: 1, precio:
       1200 },
    { producto_id: 20, nombre: "Mouse", cantidad: 2, precio: 25 }
  ]
}
```

Capítulo 8

Casos Prácticos Integradores (Ejercicios Resueltos)

En este capítulo se presentan ejercicios que combinan varios conceptos de la sesión.

8.1. Ejercicio 1: Banca en Línea

Requisitos: Diseñar una base de datos para un sistema bancario con cuentas, clientes, transacciones (depósitos, retiros, transferencias) y sucursales. Considerar concurrencia, particionamiento y posibles cuellos de botella.

Solución paso a paso:

1. Modelo conceptual: Entidades Cliente, Cuenta, Transaccion, Sucursal.
2. Modelo lógico:
   - Cliente (id_cliente, nombre, direccion, telefono)
   - Sucursal (id_sucursal, nombre, ciudad)
   - Cuenta (id_cuenta, id_cliente, id_sucursal, tipo, saldo, fecha_apertura)
   - Transaccion (id_trans, id_cuenta, tipo, monto, fecha, id_cuenta_destino nullable)
3. Consideraciones de concurrencia: Al actualizar saldos, usar transacciones y niveles de aislamiento adecuados (READ COMMITTED). Para evitar actualizaciones perdidas, se puede usar SELECT FOR UPDATE.
4. Particionamiento: Por ejemplo, particionar Transaccion por fecha (rango mensual) para mejorar rendimiento de consultas históricas.
5. Índices: índice en Transaccion(id_cuenta, fecha) para consultas de movimientos.
6. Implementación de transferencia: Usar una transacción que reste de una cuenta y sume a otra, con comprobación de saldo.

8.2. Ejercicio 2: Data Warehouse de Ventas

Problema: Diseñar un data warehouse para una cadena de supermercados. Se desea analizar ventas por producto, tienda, tiempo y promoción. Incluir dimensiones lentamente cambiantes (SCD tipo 2 para precios de productos).

Solución:

1. Dimensiones: Tiempo, Producto (SCD tipo 2), Tienda, Promoción.
2. Tabla de hechos: Ventas (id_tiempo, id_producto, id_tienda, id_promocion, cantidad, importe).
3. Para SCD tipo 2 en Producto, la tabla producto tendrá: id_producto, nombre, categoria, precio, fecha_inicio, fecha_fin, activo. Cada vez que cambia el precio, se inserta una nueva fila.
4. ETL: Extraer de sistemas transaccionales, transformar (calcular importe, asignar claves de dimensión) y cargar.

8.3. Ejercicio 3: Trigger de Histórico

Problema: Crear un trigger que mantenga un histórico de cambios en la tabla `empleados` (salario y departamento) en una tabla `historial_empleado`.

Solución:

```
CREATE TABLE historial_empleado (
    id_empleado INT,
    salario_anterior DECIMAL,
    salario_nuevo DECIMAL,
    depto_anterior INT,
    depto_nuevo INT,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    usuario TEXT DEFAULT current_user
);

CREATE OR REPLACE FUNCTION historial_empleado_func()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO historial_empleado (id_empleado, salario_anterior
        , salario_nuevo, depto_anterior, depto_nuevo)
    VALUES (OLD.id, OLD.salario, NEW.salario, OLD.departamento_id
        , NEW.departamento_id);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trig_historial_empleado
AFTER UPDATE OF salario, departamento_id ON empleados
FOR EACH ROW EXECUTE FUNCTION historial_empleado_func();
```

8.4. Ejercicio 4: Comparación SQL vs MongoDB

Problema: Para un blog con posts, comentarios y likes, modelar en SQL y MongoDB. Discutir ventajas y desventajas.

Solución SQL:

- Tablas: Post, Comentario, Like (con FK a post y usuario).
- Consultas: Para mostrar un post con sus comentarios y número de likes, se requieren joins.

Solución MongoDB:

- Colección posts: cada documento contiene array de comentarios (embebidos) y un contador de likes.
- Si los likes son muchos, podrían ir en una colección separada.
- Ventaja: una sola consulta devuelve todo el post con comentarios.
- Desventaja: si los comentarios son muy numerosos, puede exceder el tamaño máximo del documento (16MB en MongoDB). En ese caso, conviene referenciar.

Glosario

ACID Conjunto de propiedades (Atomicidad, Consistencia, Aislamiento, Durabilidad) que garantizan transacciones confiables.

Clave de partición En bases de datos distribuidas, determina el nodo donde se almacena una fila.

Clustering Grupo de servidores que trabajan juntos para alta disponibilidad y escalabilidad.

Data Warehouse Almacén de datos orientado a análisis, con datos históricos y modelado dimensional.

Deadlock Situación donde dos o más transacciones se bloquean mutuamente.

ETL Proceso de Extracción, Transformación y Carga de datos.

Fragmentación División de una tabla en partes almacenadas en diferentes nodos.

Índice cubriente Índice que contiene todas las columnas necesarias para una consulta, evitando acceder a la tabla.

MVCC Multi-Version Concurrency Control, técnica que permite alta concurrencia sin bloqueos de lectura.

OLAP Procesamiento analítico en línea, orientado a consultas complejas.

OLTP Procesamiento transaccional en línea, orientado a muchas transacciones cortas.

Replicación Copia de datos en múltiples servidores para disponibilidad o balanceo.

SCD Slowly Changing Dimension, dimensión que cambia lentamente con el tiempo.

Sharding Técnica de particionamiento horizontal a gran escala.

Trigger Procedimiento que se ejecuta automáticamente ante un evento en una tabla.

Vista materializada Vista que almacena físicamente el resultado de una consulta.

Referencias

- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems*. 7th ed. Pearson.
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts*. 7th ed. McGraw-Hill.
- Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit*. 3rd ed. Wiley.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly.
- Documentación oficial de PostgreSQL: `https://www.postgresql.org/docs/`
- Documentación de MongoDB: `https://docs.mongodb.com/`
- Documentación de Apache Cassandra: `https://cassandra.apache.org/doc/`